1. Data Types in Python

Data Type बताता है कि किसी variable में किस प्रकार का data stored है।

Common Python Data Types

Data Type	Example	Use
int	10	Whole number
float	10.5	Decimal number
complex	5+2j	Complex number
str	"Python"	Text
list	[1,2,3]	Collection
tuple	(1,2,3)	Fixed collection
set	{1,2,3}	Unique collection
dict	{"name":"Vijay"}	Key-value data

---

2. Number Data Types

Python में mainly तीन प्रकार के numeric data types होते हैं:

1. int
2. float
3. complex

2.1 Integer (int)

Integer का मतलब **whole number**, यानी बिना decimal वाला number।

Examples

x = 10

y = -25

z = 0

print(x)

print(y)

print(z)

Output

10

-25

0

Real-Life Example

किसी student के marks:

marks = 85

यह int है क्योंकि इसमें decimal नहीं है।

2.2 Float (float)

Float का उपयोग **decimal numbers** के लिए होता है।

Example

price = 99.50

height = 5.8

temperature = 36.5

print(price)

print(height)

print(temperature)

Output:

99.5

5.8

36.5

याद रखो

10 → int

10.5 → float

2.3 Complex (complex)

Complex number में दो parts होते हैं:

Real Part + Imaginary Part

Python में imaginary part के लिए **j** use किया जाता है।

Example

```
x = 5 + 2j


print(x)
```

Output:

```
(5+2j)
```

यहाँ:

5 → Real part

2j → Imaginary part

---

3. Arithmetic Operations

Numbers पर mathematical operations कर सकते हैं।

Operator	Meaning	Example
+	Addition	10 + 5
-	Subtraction	10 - 5
*	Multiplication	10 * 5
/	Division	10 / 5
//	Floor Division	10 // 3
%	Modulus	10 % 3
**	Power	2 ** 3

Example

```
a = 10
b = 3


print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
```

print(a % b)

print(a ** b)

---

4. String

Definition

String is a sequence of characters enclosed within single (' ') or double (" ") quotes.

Example:

name = "Vijay"

यहाँ "Vijay" एक string है।

String में क्या-क्या हो सकता है?

"Python"

"Hello World"

"12345"

"Python123"

"@#$%"

ध्यान दो:

"123"

यह **string** है।

लेकिन:

123

यह **integer** है।

---

5. Creating a String

Single Quotes

name = 'Vijay'

print(name)

Double Quotes

name = "Vijay"

print(name)

दोनों valid हैं।

---

6. Multi-line String

Multiple lines के लिए triple quotes use कर सकते हैं।

message = """Hello

My name is Vijay

I am learning Python"""

```
print(message)
```

Output:

```
Hello
My name is Vijay
I am learning Python
```

---

7. String Indexing

String के प्रत्येक character का एक **index number** होता है।

Index हमेशा **0 से शुरू** होता है।

String: P y t h o n

Index: 0 1 2 3 4 5

Example

```
s = "Python"


print(s[0])
print(s[1])
print(s[5])
```

Output:

```
P
```

y

n

Important

First character → index 0

Second character → index 1

---

8. Negative Indexing

Python में negative index भी होता है।

String: P y t h o n

Positive: 0 1 2 3 4 5

Negative: -6 -5 -4 -3 -2 -1

Example

```
s = "Python"


print(s[-1])

print(s[-2])
```

Output:

n

o

याद रखो

-1 हमेशा **last character** को represent करता है।

---

9. String Slicing

String का कुछ हिस्सा निकालने के लिए **slicing** use करते हैं।

Syntax

string[start : stop]

stop index include नहीं होता।

Example

s = "Python"

print(s[0:3])

Output:

Pyt

क्योंकि indexes:

0 → P

1 → y

2 → t

3 → h

3 include नहीं हुआ।

Different Slicing Examples

s = "Python"

print(s[:3])

print(s[2:])

print(s[:])

Output:

Pyt

thon

Python

---

10. String Concatenation

दो या अधिक strings को जोड़ने के लिए + operator use होता है।

Example

first = "Hello"

second = "World"

result = first + " " + second

print(result)

Output:

Hello World

याद रखो

"Hello" + "World"

Output:

HelloWorld

Space चाहिए तो:

"Hello" + " " + "World"

---

11. String Repetition

* operator से string को multiple times repeat कर सकते हैं।

s = "Python "

print(s * 3)

Output:

Python Python Python

---

12. Important String Methods

अब book में दिए गए important string methods को समझते हैं।

12.1 islower()

यह check करता है कि string में कम-से-कम एक cased character है और सभी cased characters lowercase हैं या नहीं।

Syntax

string.islower()

Example

s = "Python Programming"

print(s.islower())

s = "python programming"

print(s.islower())

Output:

False

True

याद रखो

python → True

Python → False

---

13. isupper()

Check करता है कि string में कम-से-कम एक cased character है और सभी cased characters uppercase हैं या नहीं।

Example

s = "Python Programming"

print(s.isupper())

s = "PYTHON PROGRAMMING"

print(s.isupper())

Output:

False

True

---

14. isnumeric()

Check करता है कि string में केवल numeric characters हैं या नहीं।

Example

```
s = "Python Programming123"

print(s.isnumeric())


s = "12345"

print(s.isnumeric())
```

Output:

False

True

Important

"12345".isnumeric()

→ True

लेकिन:

"123abc".isnumeric()

→ False

---

15. isspace()

Check करता है कि string में केवल whitespace characters हैं या नहीं।

Whitespace के examples:

space

tab

newline

Example

```
s = "Python Programming"
print(s.isspace())

s = "   "
print(s.isspace())
```

Output:

```
False
True
```

---

16. istitle()

Check करता है कि string **title case** में है या नहीं।

Title case में प्रत्येक word का पहला character uppercase होता है।

Example

```
s = "Python programming is fun"

print(s.istitle())


s = "Python Programming Is Fun"

print(s.istitle())
```

Output:

False

True

---

17. expandtabs()

String में मौजूद tab character \t को spaces में convert करता है।

Syntax

```
string.expandtabs(tabsize=8)
```

Default tab size:

8

Example

```
s = "Python\tProgramming\tis\tfun"


print(s.expandtabs())
```

print(s.expandtabs(10))

यह \t को दिए गए tab size के अनुसार spaces में expand करेगा।

Important

\t → Tab

---

18. join()

join() sequence के elements को किसी separator की मदद से जोड़ता है।

Syntax

separator.join(sequence)

Example

s = "-"

seq = ("Python", "Programming")

print(s.join(seq))

Output:

Python-Programming

दूसरा Example

```
s = "*"

seq = ("Python", "Programming")

print(s.join(seq))
```

Output:

Python*Programming

Real-Life Example

Names को comma से जोड़ना:

```
names = ["Ram", "Shyam", "Mohan"]

result = ", ".join(names)

print(result)
```

Output:

Ram, Shyam, Mohan

---

19. split()

String को parts में divide करके **list** return करता है।

Syntax

string.split(separator)

अगर separator नहीं दिया गया, तो whitespace के आधार पर split करता है।

Example

s = "Python Programming is fun"

print(s.split())

Output:

['Python', 'Programming', 'is', 'fun']

Separator के साथ split()

s = "Python*Programming*is*fun"

print(s.split("*"))

Output:

['Python', 'Programming', 'is', 'fun']

---

split() में limit

s = "Python*Programming*is*fun"

print(s.split("*", 2))

Output:

['Python', 'Programming', 'is*fun']

यहाँ maximum **2 splits** हुए।

---

20. splitlines()

String को newline (\n) के आधार पर split करता है और list return करता है।

Example

s = "Python\nProgramming\nis\nfun"

print(s.splitlines())

Output:

['Python', 'Programming', 'is', 'fun']

\n का मतलब

\n → New Line

🧠 String Methods — Quick Revision

Method	काम
islower()	Check lowercase
isupper()	Check uppercase
isnumeric()	Check numeric characters
isspace()	Check whitespace
istitle()	Check title case
expandtabs()	Tab को spaces में बदलता है
join()	Strings को separator से जोड़ता है
split()	String को list में divide करता है
splitlines()	Newline के आधार पर split करता है

1. LIST

1.1 List क्या है?

Definition:
A list is an ordered collection of items. It is one of the most commonly used and flexible data types in Python.

List को **square brackets [ ]** के अंदर लिखा जाता है।

my_list = [10, 20, 30, 40]

List की मुख्य विशेषताएँ

- Ordered होती है
- Duplicate values रख सकती है
- अलग-अलग data types रख सकती है
- Mutable होती है यानी values change कर सकते हैं
- Indexing और slicing support करती है

Example

my_list = ['abcd', 147, 2.43, 'Tom', 74.9]

print(my_list)

Output:

['abcd', 147, 2.43, 'Tom', 74.9]

एक ही list में:

String + Integer + Float

हो सकते हैं।

---

2. List Indexing

List में index **0 से शुरू** होता है।

List: 10 20 30 40

Index: 0 1 2 3

Example

```
numbers = [10, 20, 30, 40]


print(numbers[0])
print(numbers[2])
```

Output:

10

30

---

3. List Slicing

List के कुछ elements निकालने के लिए slicing करते हैं।

Syntax

```
list[start:stop]
```

Example

```
numbers = [10, 20, 30, 40, 50]
```

print(numbers[1:4])

Output:

[20, 30, 40]

stop index include नहीं होता।

---

4. List में Different Types

first_list = ['abcd', 147, 2.43, 'Tom', 74.9]

print(first_list)

print(first_list[0])

print(first_list[1:3])

print(first_list[2:])

Output:

['abcd', 147, 2.43, 'Tom', 74.9]

abcd

[147, 2.43]

[2.43, 'Tom', 74.9]

---

5. List Repetition

* operator से list को repeat कर सकते हैं।

small_list = [111, 'Tom']

print(small_list * 2)

Output:

[111, 'Tom', 111, 'Tom']

---

6. List Concatenation

दो lists को + से जोड़ सकते हैं।

list1 = [1, 2, 3]

list2 = [4, 5, 6]

print(list1 + list2)

Output:

[1, 2, 3, 4, 5, 6]

---

7. Updating a List

List **mutable** होती है, इसलिए इसके elements को change कर सकते हैं।

```
my_list = ['abcd', 147, 2.43, 'Tom', 74.9]


print(my_list)


my_list[2] = 500


print(my_list)
```

Output:

```
['abcd', 147, 2.43, 'Tom', 74.9]


['abcd', 147, 500, 'Tom', 74.9]
```

Multiple elements update

```
my_list[0] = 20

my_list[1] = 'apple'
```

---

8. Deleting List Elements

List से element delete करने के लिए del statement use कर सकते हैं।

```
my_list = ['abcd', 147, 2.43, 'Tom', 74.9]


del my_list[2]


print(my_list)
```

Output:

```
['abcd', 147, 'Tom', 74.9]
```

---

9. Built-in List Functions

9.1 len(list)

List में total elements की संख्या देता है।

```
numbers = [10, 20, 30, 40]


print(len(numbers))
```

Output:

```
4
```

9.2 max(list)

List की maximum value देता है।

numbers = [1200, 147, 2.43, 1.12]

print(max(numbers))

Output:

1200

9.3 min(list)

List की minimum value देता है।

numbers = [1200, 147, 2.43, 1.12]

print(min(numbers))

Output:

1.12

9.4 list(sequence)

किसी sequence को list में convert करता है।

```
my_tuple = ('abcd', 147, 2.43, 'Tom')


print(list(my_tuple))
```

Output:

['abcd', 147, 2.43, 'Tom']

---

10. map() with List

map() का उपयोग किसी function को sequence के प्रत्येक element पर apply करने के लिए किया जाता है।

Example

```
data = input("Enter numbers: ")


numbers = list(map(int, data.split()))


print(numbers)
```

अगर input:

1 2 3 4

तो output:

[1, 2, 3, 4]

समझो:

input()

↓

"1 2 3 4"

↓

split()

↓

["1", "2", "3", "4"]

↓

map(int, ...)

↓

1, 2, 3, 4

↓

list()

↓

[1, 2, 3, 4]

---

11. Built-in List Methods

अब list के important methods:

11.1 append()

List के **end में एक element** add करता है।

Syntax

list.append(object)

Example

my_list = ['abcd', 147, 2.43, 'Tom']

my_list.append(100)

print(my_list)

Output:

['abcd', 147, 2.43, 'Tom', 100]

याद रखो

append() → **एक element end में add**

---

12. count()

किसी element की list में occurrence/count बताता है।

my_list = ['abcd', 147, 2.43, 'Tom', 147, 200, 147]

print(my_list.count(147))

Output:

3

---

13. remove()

किसी specified element को list से remove करता है।

my_list = ['abcd', 147, 2.43, 'Tom']

my_list.remove('Tom')

print(my_list)

Output:

['abcd', 147, 2.43]

Important

remove() **value** के आधार पर element हटाता है।

---

14. index()

किसी element का index return करता है।

```
my_list = ['abcd', 147, 2.43, 'Tom']


print(my_list.index(2.43))
```

Output:

2

---

15. extend()

एक list के elements को दूसरी list में add करता है।

```
list1 = ['abcd', 147, 2.43, 'Tom']
list2 = ['def', 100]


list1.extend(list2)


print(list1)
```

Output:

['abcd', 147, 2.43, 'Tom', 'def', 100]

append() vs extend()

```
a = [1, 2]
a.append([3, 4])
```

Result:

[1, 2, [3, 4]]

लेकिन:

a = [1, 2]

a.extend([3, 4])

Result:

[1, 2, 3, 4]

⭐ **Very Important Interview Question**

---

16. reverse()

List के elements को reverse कर देता है।

my_list = ['abcd', 147, 2.43, 'Tom']

my_list.reverse()

print(my_list)

Output:

['Tom', 2.43, 147, 'abcd']

---

17. insert()

किसी particular index पर element insert करता है।

Syntax

list.insert(index, object)

Example

my_list = ['abcd', 147, 2.43, 'Tom']

my_list.insert(2, 222)

print(my_list)

Output:

['abcd', 147, 222, 2.43, 'Tom']

---

18. sort()

List को ascending order में sort करता है।

numbers = [890, 147, 2.43, 100]

numbers.sort()

print(numbers)

Output:

[2.43, 100, 147, 890]

---

19. pop()

List से element remove करता है और removed element return भी करता है।

Default रूप से last element remove होता है।

numbers = [10, 20, 30, 40]

x = numbers.pop()

print(x)

print(numbers)

Output:

40

[10, 20, 30]

Index देकर भी remove कर सकते हैं:

numbers = [10, 20, 30, 40]

```
numbers.pop(1)

print(numbers)
```

Output:

```
[10, 30, 40]
```

⭐ List Methods Quick Revision

Method	काम
append()	End में एक element add
count()	Element कितनी बार आया
remove()	Value remove
index()	Element का index
extend()	दूसरे sequence के elements add
reverse()	List reverse
insert()	Given index पर element add
sort()	List sort
pop()	Element remove + return

---

20. TUPLE

Tuple क्या है?

Definition:
A tuple is another sequence data type similar to a list, but unlike lists, tuples are **immutable**.

Tuple को generally parentheses () में लिखा जाता है।

my_tuple = ('abcd', 147, 2.43, 'Tom')

---

21. Tuple की Characteristics

Tuple:

- Ordered होता है
- Duplicate values रख सकता है
- Different data types रख सकता है
- Indexing support करता है
- Slicing support करता है
- **Immutable होता है**

सबसे important:

List → Mutable

Tuple → Immutable

---

22. Tuple Example

first_tuple = ('abcd', 147, 2.43, 'Tom')

print(first_tuple)

print(first_tuple[0])

print(first_tuple[1:3])

Output:

('abcd', 147, 2.43, 'Tom')

abcd

(147, 2.43)

---

23. Tuple Slicing

t = (10, 20, 30, 40, 50)

print(t[1:4])

Output:

(20, 30, 40)

---

24. Tuple Repetition

t = (1, 2)

print(t * 2)

Output:

(1, 2, 1, 2)

---

25. Tuple Concatenation

t1 = (1, 2)

t2 = (3, 4)

print(t1 + t2)

Output:

(1, 2, 3, 4)

---

26. Tuple Cannot Be Modified

t = (10, 20, 30)

t[1] = 100

यह error देगा क्योंकि tuple immutable है।

Real-life example

अगर किसी student के permanent record में:

student = ("Vijay", 20, "BCA")

और values change नहीं करनी हैं, तो tuple useful हो सकता है।

---

27. Tuple Deletion

पूरे tuple को del से delete कर सकते हैं।

t = (10, 20, 30)

del t

लेकिन individual element delete नहीं कर सकते।

❌ यह allowed नहीं:

del t[1]

---

28. Built-in Tuple Functions

len(tuple)

t = ('abcd', 147, 2.43, 'Tom')

print(len(t))

Output:

4

---

max(tuple)

t = (1200, 147, 2.43, 1.12)

print(max(t))

Output:

1200

---

min(tuple)

t = (1200, 147, 2.43, 1.12)

print(min(t))

Output:

1.12

---

tuple(sequence)

List को tuple में convert कर सकते हैं।

my_list = ['abcd', 147, 2.43, 'Tom']

print(tuple(my_list))

Output:

('abcd', 147, 2.43, 'Tom')

⭐ List vs Tuple

Feature	List	Tuple
Brackets	[ ]	( )
Ordered	✅	✅
Duplicate	✅	✅
Indexing	✅	✅
Slicing	✅	✅
Mutable	✅	❌
Elements change	✅	❌

याद रखने की trick:

List = Change कर सकते हैं

Tuple = Fixed रहता है

---

29. SET

Set क्या है?

Definition:

A set is an **unordered collection of unique elements**.

Set को curly braces { } में लिखा जाता है।

s = {1, 2, 3}

---

30. Set की Characteristics

Set:

- Unordered होता है
- Duplicate values allow नहीं करता
- Unique elements रखता है
- Indexing नहीं होती
- Slicing नहीं होती
- Mutable होता है
- Set operations कर सकते हैं

---

31. Set Example

s = {1, 2, 3}

print(s)

Output:

{1, 2, 3}

Order को rely नहीं करना चाहिए क्योंकि set unordered collection है।

---

32. Duplicate Values in Set

s = {1, 2, 3, 2, 1, 3}

print(s)

Output में केवल unique values रहेंगी:

{1, 2, 3}

⭐ **Important**

Set automatically duplicates remove कर देता है।

---

33. Set with Mixed Data Types

s = {1, 2.4, 'apple', 'Tom', 3}

print(s)

Set में different types के elements रख सकते हैं, provided elements hashable हों।

---

34. Creating Set from List

set() function से list को set में convert कर सकते हैं।

numbers = [1, 2, 3, 2, 1, 4]

s = set(numbers)

print(s)

Output:

{1, 2, 3, 4}

---

35. Empty Set

⚠️ यह बहुत important है।

s = {}

यह empty set नहीं है। यह **empty dictionary** है।

Empty set बनाने के लिए:

s = set()

---

36. Built-in Set Functions

len(set)

Set में total unique elements की संख्या देता है।

s = {'abcd', 147, 2.43, 'Tom'}

print(len(s))

Output:

4

---

max(set)

s = {1200, 147, 2.43, 1.12}

print(max(s))

Output:

1200

---

min(set)

s = {1200, 147, 2.43, 1.12}

print(min(s))

Output:

1.12

---

sum(set)

Set के सभी numeric elements का sum देता है।

s = {147, 2.43}

print(sum(s))

Output:

149.43

---

sorted(set)

Set के elements को sorted form में एक **new list** के रूप में देता है।

s = {213, 100, 289, 40, 23, 1, 1000}

result = sorted(s)

print(result)

Output:

[1, 23, 40, 100, 213, 289, 1000]

Important

sorted() original set को modify नहीं करता।

---

37. any()

अगर set में **कम-से-कम एक truthy element** है तो True return करता है।

s1 = set()

s2 = {1, 2, 3, 4}

print(any(s1))

print(any(s2))

Output:

False

True

---

38. all()

अगर सभी elements truthy हैं, तो True return करता है।

s = {1, 2, 3, 4}

print(all(s))

Output:

True

Important

Empty set पर:

all(set())

का result भी True होता है।

---

39. Set Methods

अब important set methods।

39.1 add()

Set में एक element add करता है।

s = {3, 8, 2, 6}

s.add(9)

print(s)

अब 9 set में add हो जाएगा।

Important

अगर वही element पहले से मौजूद है, duplicate नहीं बनेगा।

---

40. remove()

Set से specified element remove करता है।

s = {3, 8, 2, 6}

s.remove(8)

print(s)

Output में 8 नहीं होगा।

⚠️ अगर element मौजूद नहीं है, remove() **KeyError** दे सकता है।

---

41. discard()

Set से element remove करता है।

s = {3, 8, 2, 6}

```
s.discard(8)
print(s)

s.discard(9)
print(s)
```

अगर 9 मौजूद नहीं है, तब भी error नहीं आएगा।

remove() vs discard()

	remove()	**discard()**
Element present	Remove	Remove
Element absent	❌ Error	✅ No error

⭐ **Very Important**

---

42. Set Operations

Sets की सबसे important feature है **set operations**।

मान लो:

A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

42.1 Union

दोनों sets के **सभी unique elements**।

print(A.union(B))

Output:

{1, 2, 3, 4, 5, 6}

Operator:

A | B

---

43. Intersection

दोनों sets में जो elements common हैं।

print(A.intersection(B))

Output:

{3, 4}

Operator:

A & B

Real-life example

A = Cricket खेलने वाले students
B = Football खेलने वाले students

Intersection = जो students **दोनों खेलते हैं**।

---

44. Difference

A में हैं लेकिन B में नहीं।

print(A.difference(B))

Output:

{1, 2}

Operator:

A - B

---

45. Symmetric Difference

जो elements केवल A या केवल B में हैं, लेकिन दोनों में common नहीं हैं।

print(A.symmetric_difference(B))

Output:

{1, 2, 5, 6}

Operator:

A ^ B

---

46. Subset

Check करता है कि एक set के सभी elements दूसरे set में मौजूद हैं या नहीं।

A = {1, 2, 3, 4}

B = {1, 2}

print(B.issubset(A))

Output:

True

क्योंकि B के 1,2 दोनों A में हैं।

---

47. Superset

Check करता है कि पहला set दूसरे set के सभी elements contain करता है या नहीं।

A = {1, 2, 3, 4}

B = {1, 2}

print(A.issuperset(B))

Output:

True

याद रखने की trick

Subset:

छोटा set → बड़े set के अंदर

Superset:

बड़ा set → छोटे set को contain करता है

---

48. Frozenset

Frozenset क्या है?

Frozenset is an immutable version of a set.

Normal set:

s = {1, 2, 3}

Frozenset:

fs = frozenset({1, 2, 3})

Main difference

Set → Mutable

Frozenset → Immutable

---

49. Frozenset Example

set1 = frozenset({3, 8, 4, 6})

set2 = frozenset({3, 8})

print(set1)

print(set2)

print(set1.intersection(set2))

Output:

frozenset({3, 8, 4, 6})

frozenset({3, 8})

frozenset({3, 8})

---

50. Frozenset में क्या नहीं कर सकते?

Frozenset immutable है, इसलिए:

fs.add(10)

❌ नहीं कर सकते।

इसी तरह:

fs.remove(2)

fs.update(...)

भी नहीं कर सकते।

लेकिन set operations जैसे:

intersection()

union()

difference()

issubset()

issuperset()

कर सकते हैं।

⭐ LIST vs TUPLE vs SET

Feature	List	Tuple	Set
Syntax	[ ]	( )	{ }
Ordered	✅	✅	❌
Duplicate	✅	✅	❌
Mutable	✅	❌	✅
Indexing	✅	✅	❌
Slicing	✅	✅	❌
Unique elements	❌	❌	✅

एक line में:

List → Ordered + Mutable

Tuple → Ordered + Immutable

Set → Unordered + Unique

⭐ SET vs FROZENSET

Feature	Set	Frozenset
Mutable	✅	❌
Unique	✅	✅
Unordered	✅	✅
add()	✅	❌
remove()	✅	❌
Set operations	✅	✅
Hashable	❌	✅

1. DICTIONARY

1.1 Dictionary क्या है?

Definition:

A dictionary is an **unordered collection of key-value pairs**.

Dictionary में data **key : value** के form में store होता है।

Syntax

```
dictionary = {
   key1: value1,
```

```
    key2: value2
}
```

Example

```
student = {
    "Name": "Tom",
    "Age": 20,
    "Height": 160
}


print(student)
```

Output:

```
{'Name': 'Tom', 'Age': 20, 'Height': 160}
```

Real-life example

Student की information:

Name → Vijay

Age → 20

Course → BCA

इसे dictionary में:

```
student = {
```

```
    "Name": "Vijay",
    "Age": 20,
    "Course": "BCA"
}
```

यहाँ:

Name → Key

Vijay → Value

Age → Key

20 → Value

---

2. Dictionary में Value Access करना

Dictionary में value को **key** की मदद से access करते हैं।

```
student = {
    "Name": "Vijay",
    "Age": 20,
    "Course": "BCA"
}
```

print(student["Name"])

print(student["Age"])

Output:

Vijay

20

याद रखो

List/Tuple → Index से value

Dictionary → Key से value

---

3. Dictionary Create करना

Empty dictionary:

dict1 = {}

फिर values add कर सकते हैं:

dict1["Name"] = "Vijay"

dict1["Age"] = 20

Print:

print(dict1)

Output:

{'Name': 'Vijay', 'Age': 20}

---

4. Dictionary में Different Types

Dictionary की values अलग-अलग types की हो सकती हैं।

```
student = {
   "Name": "Vijay",
   "Age": 20,
   "Marks": 85.5,
   "Passed": True
}

print(student)
```

यहाँ values हैं:

String

Integer

Float

Boolean

---

5. keys() Method

keys() dictionary की सभी keys return करता है।

```
student = {
   "Name": "Vijay",
   "Age": 20,
   "Course": "BCA"
}

print(student.keys())
```

Output:

dict_keys(['Name', 'Age', 'Course'])

---

6. values() Method

values() dictionary की सभी values return करता है।

```
print(student.values())
```

Output:

dict_values(['Vijay', 20, 'BCA'])

---

7. items() Method

items() सभी **key-value pairs** देता है।

```
print(student.items())
```

Output:

```
dict_items([
   ('Name', 'Vijay'),
   ('Age', 20),
   ('Course', 'BCA')
])
```

---

8. Dictionary को Update करना

Dictionary mutable होती है, इसलिए existing value को change कर सकते हैं।

```
student = {
   "Name": "Tom",
   "Age": 20,
   "Height": 160
}

student["Age"] = 25

print(student)
```

Output:

{'Name': 'Tom', 'Age': 25, 'Height': 160}

Important

अगर key पहले से मौजूद है:

student["Age"] = 25

तो पुरानी value update होगी।

---

9. Dictionary में New Key-Value Add करना

अगर key मौजूद नहीं है, तो नई key-value pair add हो जाएगी।

```
student = {
   "Name": "Tom",
   "Age": 20
}


student["Weight"] = 60


print(student)
```

Output:

{'Name': 'Tom', 'Age': 20, 'Weight': 60}

इसलिए एक ही syntax दो काम करता है:

dict[key] = value

Existing key → Update

New key → Add

---

10. Dictionary से Element Delete करना

del statement से individual key-value pair delete कर सकते हैं।

```
student = {
   "Name": "Tom",
   "Age": 20,
   "Height": 160
}

del student["Age"]

print(student)
```

Output:

{'Name': 'Tom', 'Height': 160}

---

11. clear()

पूरी dictionary के elements remove कर देता है।

```
student = {
   "Name": "Tom",
   "Age": 20,
   "Height": 160
}

student.clear()

print(student)
```

Output:

```
{}
```

Difference

del student

→ dictionary object ही delete हो सकता है।

```
student.clear()
```

→ dictionary रहती है लेकिन उसके अंदर के सारे elements हट जाते हैं।

---

12. Properties of Dictionary Keys

Property 1: Duplicate Keys Allowed नहीं हैं

एक dictionary में same key multiple times नहीं हो सकती।

```
student = {
   "Name": "Tom",
   "Name": "Vijay"
}

print(student)
```

Final assignment की value रखी जाएगी:

{'Name': 'Vijay'}

याद रखो

Dictionary में **keys unique** होती हैं।

---

13. Dictionary Keys Immutable होनी चाहिए

Dictionary की keys generally **hashable/immutable objects** होनी चाहिए।

Valid keys:

```
{
   "Name": "Vijay",
   1: "One",
   (1, 2): "Pair"
}
```

List जैसी mutable object key नहीं हो सकती:

```
{
   [1, 2]: "Hello"
}
```

❌ Error आएगा।

---

14. Built-in Dictionary Functions

14.1 len(dict)

Dictionary में total key-value pairs की संख्या देता है।

```
student = {
   "Name": "Tom",
```

```
    "Age": 20,
    "Height": 160
}

print(len(student))
```

Output:

3

---

15. str(dict)

Dictionary का string representation देता है।

```
student = {
    "Name": "Tom",
    "Age": 20
}

print(str(student))
```

Output:

{'Name': 'Tom', 'Age': 20}

---

16. type(variable)

किसी variable का data type बताता है।

```
student = {
   "Name": "Tom",
   "Age": 20
}

print(type(student))
```

Output:

```
<class 'dict'>
```

Example:

```
name = "Vijay"
numbers = [1, 2, 3]

print(type(name))
print(type(numbers))
```

Output:

```
<class 'str'>
<class 'list'>
```

---

17. Important Dictionary Methods

17.1 dict.clear()

सभी elements remove करता है।

```
student = {"Name": "Tom", "Age": 20}


student.clear()


print(student)
```

Output:

```
{}
```

---

18. dict.copy()

Dictionary की copy बनाता है।

```
student = {
   "Name": "Tom",
   "Age": 20
}
```

```
new_student = student.copy()


print(new_student)
```

Output:

```
{'Name': 'Tom', 'Age': 20}
```

क्यों useful है?

अगर हमें original dictionary को अलग रखते हुए उसकी copy पर काम करना हो।

---

19. dict.get(key)

किसी key की value प्राप्त करता है।

```
student = {
   "Name": "Tom",
   "Age": 20
}


print(student.get("Age"))
```

Output:

```
20
```

अगर key मौजूद नहीं है:

```
print(student.get("Phone"))
```

Output:

None

---

20. get() with Default Value

```
student = {
   "Name": "Tom",
   "Age": 20
}

print(student.get("Phone", 0))
```

Output:

0

इसका मतलब:

अगर "Phone" key नहीं मिली तो 0 return करो।

---

21. setdefault()

setdefault() key की value return करता है।

अगर key मौजूद है, तो उसकी existing value return होगी।

```
student = {
   "Name": "Tom",
   "Age": 20
}

print(student.setdefault("Age"))
```

Output:

20

Key मौजूद नहीं हो तो

```
student = {
   "Name": "Tom",
   "Age": 20
}

print(student.setdefault("Phone", 0))

print(student)
```

Output:

0

{'Name': 'Tom', 'Age': 20, 'Phone': 0}

Difference

get()

↓

Missing key → default return

लेकिन dictionary में key add नहीं करता।

setdefault()

↓

Missing key → default value return

और key-value pair dictionary में add करता है।

⭐ यह interview में पूछा जा सकता है।

---

22. dict.fromkeys()

दिए गए keys से एक नई dictionary बनाता है।

keys = ["Name", "Age", "Height"]

```
student = dict.fromkeys(keys)


print(student)
```

Output:

```
{'Name': None, 'Age': None, 'Height': None}
```

Default value भी दे सकते हैं:

```
student = dict.fromkeys(keys, 0)


print(student)
```

Output:

```
{'Name': 0, 'Age': 0, 'Height': 0}
```

⭐ Dictionary Quick Revision

Function/Method	काम
len()	Number of key-value pairs
str()	String representation
type()	Data type
keys()	सभी keys
values()	सभी values
items()	Key-value pairs
get()	Value प्राप्त करना
setdefault()	Value प्राप्त/add करना
copy()	Dictionary copy
clear()	सभी elements remove
fromkeys()	Keys से dictionary बनाना
del	Particular key delete

---

23. MUTABLE AND IMMUTABLE OBJECTS

यह chapter का बहुत important concept है।

Mutable क्या होता है?

जिस object की value को **बनाने के बाद change किया जा सकता है**, उसे mutable object कहते हैं।

Example

```
numbers = [1, 2, 3]


numbers.append(4)


print(numbers)
```

Output:

[1, 2, 3, 4]

Original list change हो गई।

इसलिए:

List → Mutable

---

24. Immutable क्या होता है?

जिस object की value बनने के बाद directly change नहीं की जा सकती, उसे **immutable** कहते हैं।

Examples:

int

float

string

tuple

Example

x = 10

अगर हम:

x = x - 2

करते हैं, तो existing integer object को modify नहीं किया जाता; नया value/object assignment होता है।

---

25. String Mutable है या Immutable?

String immutable है।

name = "abcde"

name = name.upper()

print(name)

Output:

ABCDE

यहाँ original string को modify नहीं किया गया; नया string value बनाई गई।

---

26. List Mutable है

my_list = [1, 2, 3, 4, 5]

print(my_list)

my_list.remove(3)

print(my_list)

Output:

[1, 2, 3, 4, 5]

[1, 2, 4, 5]

Original list change हो गई।

---

27. Main Mutable vs Immutable Table

Data Type	Mutable?
Integer	❌ Immutable
Float	❌ Immutable
String	❌ Immutable
Tuple	❌ Immutable
List	✅ Mutable
Set	✅ Mutable
Dictionary	✅ Mutable
Frozenset	❌ Immutable

Easy Trick

LIST + SET + DICTIONARY

↓

MUTABLE

NUMBER + STRING + TUPLE + FROZENSET

↓

IMMUTABLE

---

28. DATA TYPE CONVERSION

Data Type Conversion क्या है?

एक data type को दूसरे data type में convert करने को **data type conversion** कहते हैं।

Example:

```
x = "100"


x = int(x)


print(x)
```

अब "100" string से 100 integer बन गया।

---

29. int()

किसी suitable value को integer में convert करता है।

```
x = 12.8


print(int(x))
```

Output:

12

String to Integer

x = "100"

print(int(x))

Output:

100

---

30. float()

Value को floating-point number में convert करता है।

x = 12

print(float(x))

Output:

12.0

---

31. str()

किसी value को string में convert करता है।

x = 12

```
print(str(x))

print(type(str(x)))
```

Output:

```
12

<class 'str'>
```

---

32. complex()

Complex number बनाने के लिए use होता है।

```
real = 5

imag = 2


x = complex(real, imag)


print(x)
```

Output:

```
(5+2j)
```

Complex number का basic form:

a + bj

जहाँ:

- a = real part
- b = imaginary part

---

33. repr()

Object का expression/string representation देता है।

x = 12

print(repr(x))

Output:

12

String के साथ difference ज्यादा clearly दिख सकता है:

name = "Vijay"

print(str(name))

print(repr(name))

repr() generally debugging/developer-oriented representation देने के लिए useful है।

---

34. tuple()

किसी iterable को tuple में convert करता है।

numbers = [1, 2, 3, 4]

print(tuple(numbers))

Output:

(1, 2, 3, 4)

---

35. list()

किसी iterable को list में convert करता है।

numbers = (1, 2, 3, 4)

print(list(numbers))

Output:

[1, 2, 3, 4]

---

36. set()

Iterable को set में convert करता है।

numbers = [1, 2, 2, 3, 3]

print(set(numbers))

Output:

{1, 2, 3}

Duplicates automatically remove हो गए।

---

37. dict()

Suitable key-value pairs से dictionary create कर सकता है।

```
data = [
   ("Name", "Vijay"),
   ("Age", 20)
]

student = dict(data)

print(student)
```

Output:

{'Name': 'Vijay', 'Age': 20}

---

38. frozenset()

Set का immutable version बनाता है।

numbers = [1, 2, 3]

fs = frozenset(numbers)

print(fs)

Output:

frozenset({1, 2, 3})

---

39. chr()

Integer को corresponding character में convert करता है।

print(chr(65))

Output:

A

---

40. ord()

Character को उसके Unicode integer value में convert करता है।

print(ord('A'))

Output:

65

याद रखने की trick

chr() → Number → Character

ord() → Character → Number

---

41. hex()

Integer को hexadecimal representation में convert करता है।

print(hex(10))

Output:

0xa

---

42. oct()

Integer को octal representation में convert करता है।

print(oct(10))

Output:

0o12

---

43. eval()

String को Python expression के रूप में evaluate करता है।

Example:

x = "10 + 20"

print(eval(x))

Output:

30

⚠️ **Important:** eval() को untrusted/user-provided input पर use नहीं करना चाहिए, क्योंकि यह arbitrary Python code execute कर सकता है।

---

44. type() — Data Type Check

```
x = 100

y = "Hello"

z = [1, 2, 3]


print(type(x))

print(type(y))

print(type(z))
```

Output:

```
<class 'int'>

<class 'str'>

<class 'list'>
```

⭐ Important Data Conversion Table

Function	Conversion
int(x)	→ Integer
float(x)	→ Float
complex(x)	→ Complex
str(x)	→ String
repr(x)	→ Representation string
tuple(x)	→ Tuple
list(x)	→ List
set(x)	→ Set
dict(x)	→ Dictionary
frozenset(x)	→ Frozenset
chr(x)	Integer → Character
ord(x)	Character → Integer
hex(x)	Integer → Hexadecimal
oct(x)	Integer → Octal
type(x)	Type बताता है

📚 CHAPTER 4 — IMPORTANT DEFINITIONS

1. Number

A number is a data type used to store numeric values such as integers, floating-point numbers and complex numbers.

2. String

A string is a sequence of characters enclosed within single, double, or triple quotes.

3. List

A list is an ordered and mutable collection of elements that can contain duplicate values.

4. Tuple

A tuple is an ordered and immutable collection of elements that can contain duplicate values.

5. Set

A set is an unordered and mutable collection of unique elements.

6. Frozenset

A frozenset is an immutable version of a set whose elements cannot be changed after creation.

7. Dictionary

A dictionary is a mutable collection of key-value pairs where each key is unique.

8. Key

A key is an identifier used to access a particular value in a dictionary.

9. Value

A value is the data associated with a key in a dictionary.

10. Mutable Data Type

A mutable data type is a data type whose contents can be changed after the object is created.

Examples: List, Set, Dictionary.

11. Immutable Data Type

An immutable data type is a data type whose contents cannot be changed after the object is created.

Examples: Number, String, Tuple, Frozenset.

12. Data Type Conversion

Data type conversion is the process of converting a value from one data type to another.

Example:

```
x = int("10")
```

13. Indexing

Indexing is the process of accessing an individual element of a sequence using its position.

14. Slicing

Slicing is the process of extracting a part of a sequence using a range of indexes.

15. Hashable

A hashable object is an object whose value has a fixed hash value during its lifetime and can generally be used as a dictionary key or set element.

16. append()

append() is a list method used to add one element at the end of a list.

17. extend()

extend() is a list method used to add multiple elements from an iterable to the end of a list.

18. remove()

remove() is a method used to remove a specified element from a list or set.

19. discard()

discard() is a set method used to remove an element without raising an error if the element is absent.

20. keys()

keys() is a dictionary method that returns the keys of a dictionary.

21. values()

values() is a dictionary method that returns the values of a dictionary.

22. items()

items() is a dictionary method that returns the key-value pairs of a dictionary.

23. get()

get() is a dictionary method used to safely retrieve the value associated with a key.

24. setdefault()

setdefault() is a dictionary method that returns a key's value and adds the key with a default value if the key does not exist.

25. chr()

chr() is a built-in function that converts a Unicode code point into its corresponding character.

26. ord()

ord() is a built-in function that returns the Unicode code point of a character.

27. hex()

hex() is a built-in function that converts an integer into its hexadecimal representation.

28. oct()

oct() is a built-in function that converts an integer into its octal representation.

29. input()

input() is a built-in function used to take input from the user. It returns the input as a string.

30. eval()

eval() is a built-in function that evaluates a Python expression given as a string. It should not be used with untrusted input.